

李嘉奇

在职, 2周内到岗 | 五年工作经验 | 5 | 8

15201582721 | lijq1103@163.com | [前往在线版本](#)

工作经历

小红书 - 前端开发工程师

2022-08-11 ~ 至今 北京

- 负责IDEA大数据平台、小红书聚光平台、小红书乘风的开发与维护
- 参加公司通用图表库 Delight-Charts 的开发和维护
- 参与公司项目的讨论、需求评审、技术评审，并细化排期，确保按期交付需求
- 以前端视角对产品进行体验优化，设计需求
- 开发过程中，与每个板块负责人员保持交流沟通，确保需求按期交付

ByteDance Ltd. - 前端开发工程师

2021-08-06 ~ 2022-06-10 北京

- 负责飞轮投放平台、巨量引擎官网、千川官网以及 dou+ 的开发与维护
- 参加 Okee 组件库、橙子建站平台物料的开发与维护
- 参与公司项目的讨论、需求评审、技术评审，按期交付需求
- 以前端视角对产品进行体验优化

项目经验

IDEA 大数据平台

技术栈: Vue + axios + monorepo + Nest.js + Thrift

主要职责

- 优化整体项目架构和性能，提升系统稳定性与维护效率，确保关键业务需求按期交付。
- Code Review，确保代码质量

主要做的事情

- 架构优化: 采用 monorepo 架构拆分应用模块，减少模块间耦合
- 权限控制: 引入 RBAC-1 权限模型，实现精细化权限点控制
- 性能提升: 实施拆分子包、非核心 JS 异步加载和 CDN 缓存
- 模块重构: 重构数据洞察模块，重新设计数据结构并封装通用 hook
- 配置化升级: 抽离常量 JSON，使部分筛选项实现产品化配置
- 组件封装: 封装 Notify 下载组件和动态表单组件，支持指令式调用
- 微前端嵌入: 嵌入其他业务线模块，实现跨仓库复用功能模块

工作成果

- 整体页面加载速度提升27%，重新设计代码结构降低后续维护难度，并提高了团队协作效率和产品迭代速度。

DelightCharts 图表库

背景

- 业务上: 公司内部亟需构建一套可复用、高扩展性的通用图表库，以展示出主题相同的图表样式。
- 需求上: 图表库的可扩展性、可维护性、可复用性、性能优化、用户体验优化等。

主要职责

- 项目搭建、架构设计、数据格式设计、插件体系设计、文档优化等
- 提升图表库的扩展性、可维护性与用户体验

主要做的事情

- 项目架构: 采用 Monorepo 架构统一管理代码, 结合 Tree shaking、拆分子包技术优化包体积
- 数据格式设计: 根据 echarts 的数据结构, 抽象出 dimensions 与 metrics, 支持多种图表适配
- Hook 封装: 借鉴 Hook 思想, 封装通用 hooks 供团队调用, 提升代码复用率
- 插件体系: 提供一套插件体系, 提高图表库的可扩展性; 比如 数据转换插件、tooltip 渲染插件等
- 核心封装: 封装 chart.core 文件, 管理 ECharts 实例与事件, 逻辑清晰, 易于维护
- 用户体验: 利用 Dify 知识库 + Deepseek 大模型, 优化文档搜索体验, 精准识别用户意图, 提高搜索准确度
- 文档优化: 封装 Sandpack 功能, 实现代码在线编辑、预览、分享功能
- 前沿探索: 探索使用 ChatGPT 辅助进行单元测试与 Code Review, 优化测试覆盖率和代码质量

工作成果

- 构建出一套高效、扩展性强的图表库, 显著提升开发效率和代码维护性, 降低了项目迭代成本。
- 搭建了图表的数据格式和插件体系, 提高了代码的可维护性和可复用性。

抖音任务中心

技术栈: Next.js + JSBridge + axios + Nest.js + SequelizeORM + Thrift

主要职责

- 通过前后端协同及架构调整, 优化页面静态化、预渲染效果和多平台适配, 确保功能模块高效稳定运行。
- 参与系统性能监控, 及时发现并解决性能问题, 确保系统稳定性和安全性。

主要做的事情

- 跨端兼容: 封装统一 JSBridge 方法, 兼容 Web、Native 和 Iframe 内嵌场景
- 页面优化: 使用 SSR、SSG、ISR 技术实现页面静态化和预渲染
- 长列表优化: 利用 Suspense 与 IntersectionObserver 优化任务中心页长列表, 封装自定义 hook 降低维护成本
- 推送与适配: 实现 WebHook 站内信 push, 基于屏幕 DPI 进行低端手机分辨率适配
- 代码稳定性: 使用 Jest 进行单元测试, 提高代码稳定性和维护效率
- BFF层设计: 使用 Nest 框架聚合 RPC 接口, 封装中间件统一身份校验, 并设计接口枚举 code 降低开发成本
- 自动化与监控: 利用 Slardar 进行性能监控及异常上报, 结合飞书 SDK 实现异常提醒

工作成果

- 显著提高页面加载和响应速度, 优化了用户体验, 降低了开发和维护成本, 同时实现系统稳定性和安全性的双重提升。
- 通过前后端协同, 实现了系统的高效稳定运行, 同时提高了团队协作效率和产品迭代速度。

开源项目

FightingDesign ↗

一个 Vue3 组件库, 支持 TS、Tree shaking。